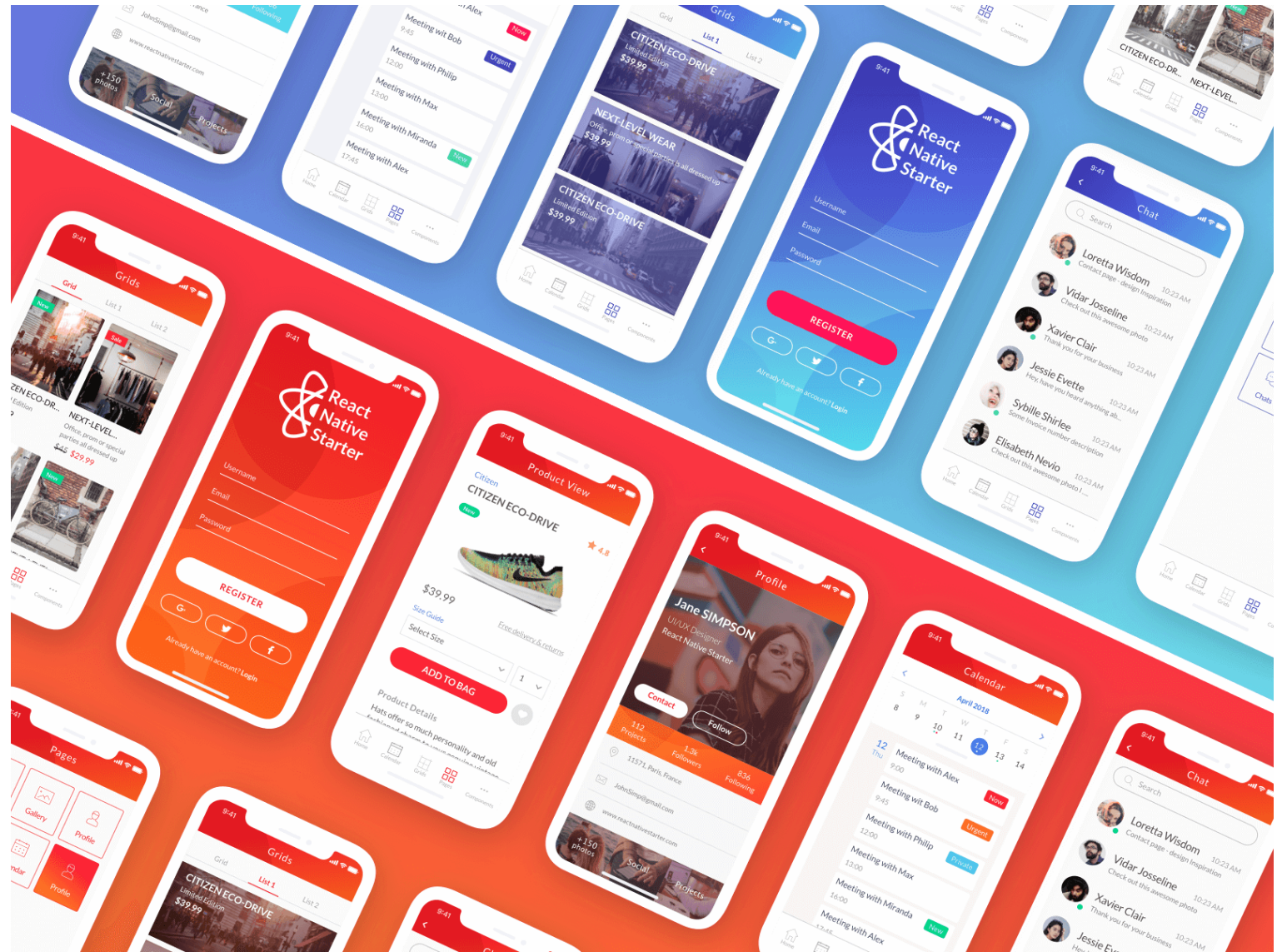


Asztali és mobil alkalmazások fejlesztése és tesztelése tantárgy – Mobil alkalmazások rész

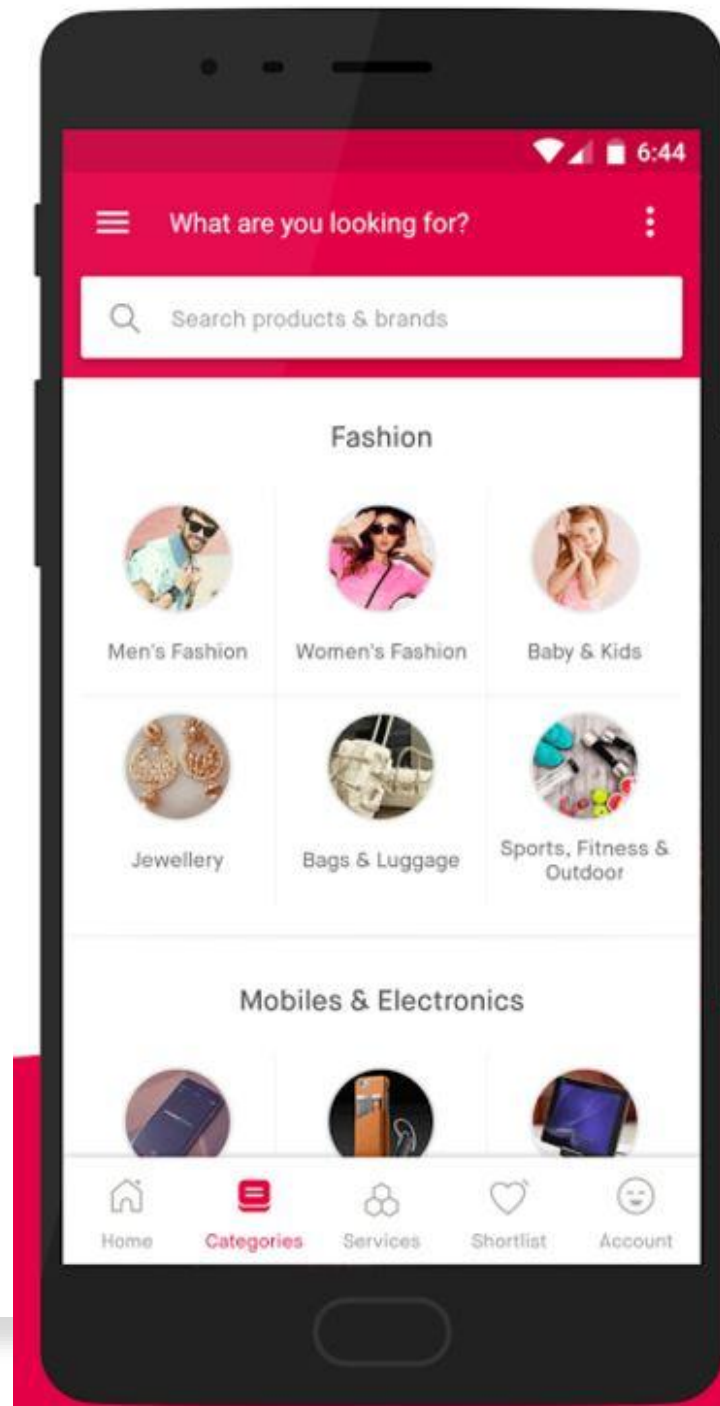
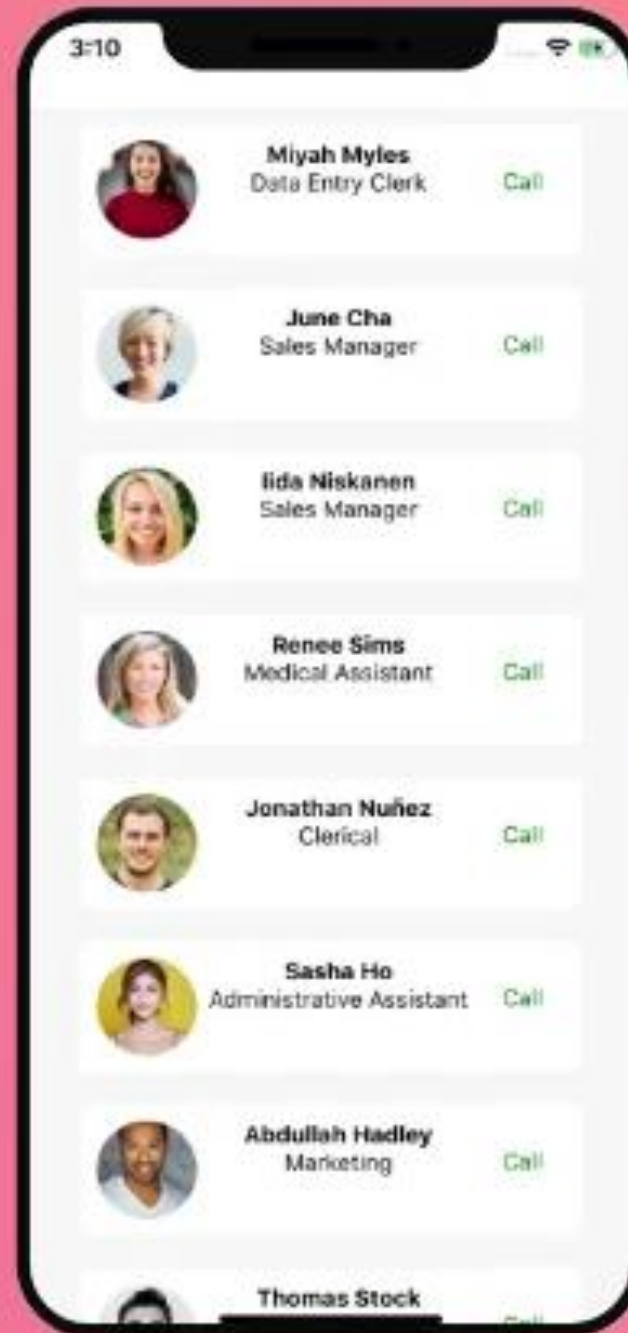
13. A

Tanárok: Dr. Koncz Adrienn
és Simon Martin

2026.01.29.



FlatList és SectionList komponensek



FlatList komponens

- A FlatList egy lista komponens, amely egyszerű, egyszintű adatszerkezetet (tömböt) jelenít meg.
- Automatikusan optimalizálja a renderelést, és csak a képernyőn látható elemeket rajzolja ki.
- Ez a leggyakrabban használt lista-komponens React Native-ban.
- <https://reactnative.dev/docs/flatlist>



FlatList komponens

Adatok beolvasása:

- A FlatList egy tömböt (array) vár:

```
const DATA = [  
  {  
    id: 'bd7acbea-c1b1-46c2-aed5-3ad53abb28ba',  
    title: 'First Item',  
  },  
  {  
    id: '3ac68afc-c605-48d3-a4f8-fbd91aa97f63',  
    title: 'Second Item',  
  },  
  {  
    id: '58694a0f-3da1-471f-bd96-145571e29d72',  
    title: 'Third Item',  
  },  
];
```

Megjelenítés módja:

Az egyes elemek megjelenítését a *renderItem* függvény határozza meg:

```
<FlatList  
  data={DATA}  
  renderItem={({item}) => <Item title={item.title} />}  
  keyExtractor={item => item.id}  
/>
```

A *keyExtractor* mindig egy egyedi kulcsot ad az elemeknek.

FlatList komponens

Kötelező és leggyakrabban használt props-ok

Props neve	Szerepe
data	A megjelenítendő adatok tömbje
renderItem	Meghatározza, hogyan jelenjen meg egy elem
keyExtractor	Egyedi kulcsot ad az elemeknek
extraData	Újrarendelést kényszerít ki állapotváltozáskor

Mikor érdemes használni a FlatList komponenst?

Ha egyszerű listát szeretnénk készíteni és megjeleníteni (pl. todo lista, kapcsolatok, hírek).

FlatList komponens

További gyakori, hasznos props-ok

Props neve	Szerepe
numColumns	Többoszlopos lista
horizontal	Vízszintes lista
ListHeaderComponent	Fejléc
ListFooterComponent	lábléc
ItemSeparatorComponent	Elválasztó vonal (itt mindig definiálni kell a vonal kinézetét is)
onRefresh, refreshing	Lehúzásos frissítés
onEndReached	Végtelen lista (scroll loading)

FlatList mintakód

First Item

Second Item

Third Item

```
import React from 'react';
import {View, FlatList, StyleSheet, Text, StatusBar} from 'react-native';
import {SafeAreaView, SafeAreaProvider} from 'react-native-safe-area-context';

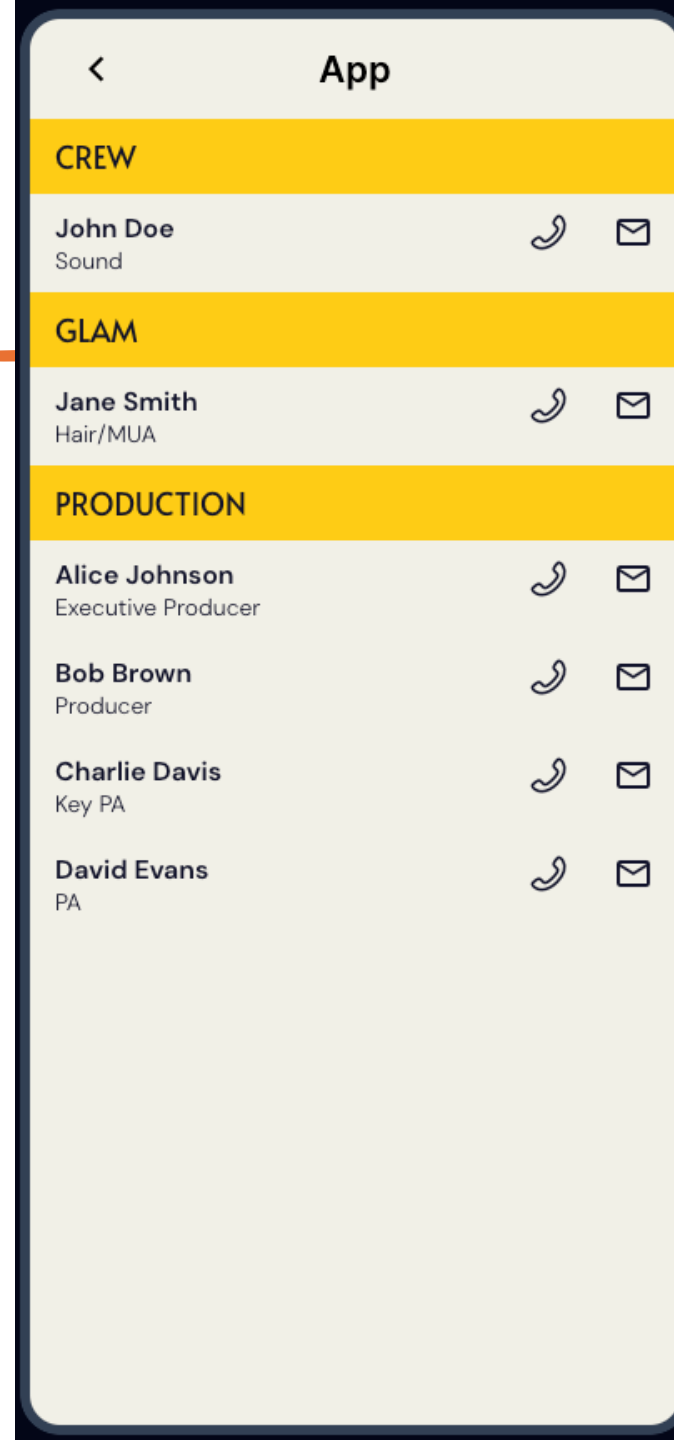
const DATA = [
  {
    id: 'bd7acbea-c1b1-46c2-aed5-3ad53abb28ba',
    title: 'First Item',
  },
  {
    id: '3ac68afc-c605-48d3-a4f8-fbd91aa97f63',
    title: 'Second Item',
  },
  {
    id: '58694a0f-3da1-471f-bd96-145571e29d72',
    title: 'Third Item',
  },
];

const Item = ({title}) => (
  <View style={styles.item}>
    <Text style={styles.title}>{title}</Text>
  </View>
);

const App = () => (
  <SafeAreaProvider>
    <SafeAreaView style={styles.container}>
      <FlatList
        data={DATA}
        renderItem={({item}) => <Item title={item.title} />}
        keyExtractor={item => item.id}
      />
    </SafeAreaView>
  </SafeAreaProvider>
);
```

SectionList komponens

- A SectionList olyan lista komponens, amely csoportosított adatokat jelenít meg úgy, hogy minden csoporthoz külön fejléctet rendel.
- Akkor érdemes használni, ha az adatok logikailag szekciókra oszthatók. Például étlapok, névjegyzék, különböző kategóriájú termékek stb.
- <https://reactnative.dev/docs/sectionlist>



SectionList komponens

Adatok beolvasása:

- A SectionList szekciók tömbjét várja:

```
const DATA = [  
  {  
    title: 'Main dishes',  
    data: ['Pizza', 'Burger', 'Risotto'],  
  },  
  {  
    title: 'Sides',  
    data: ['French Fries', 'Onion Rings', 'Fried Shrimps'],  
  },  
  {  
    title: 'Drinks',  
    data: ['Water', 'Coke', 'Beer'],  
  },  
  {  
    title: 'Desserts',  
    data: ['Cheese Cake', 'Ice Cream'],  
  },  
];
```

Megjelenítés módja:

Két renderelési függvényt használ:

```
sections={DATA}  
keyExtractor={({item, index}) => item + index}  
renderItem=(({item}) => (  
  <View style={styles.item}>  
    <Text style={styles.title}>{item}</Text>  
  </View>  
<< >>  
renderSectionHeader=(({section: {title}}) => (  
  <Text style={styles.header}>{title}</Text>  
<< >>
```

SectionList komponens

Kötelező és leggyakrabban használt props-ok

Props neve	Szerepe
sections	A szekciókat tartalmazó tömb
renderItem	Egy elem megjelenítése
renderSectionHeader	Szekció fejléce
keyExtractor	Egyedi kulcs az elemekhez

SectionList komponens

További gyakori props-ok

Props neve	Szerepe
stickySectionHeadersEnabled	Fixált szekciófejlécek (csak SectionList-tel kompatibilis)
ListHeaderComponent	A teljes lista fölött megjelenő fő fejléc, ami nem a szekciókhoz kötött. Ez csak egyszer jelenik meg a lista felett.
ListFooterComponent	A teljes lista alján megjelenő fő lábléc. Nem tartozik egyetlen elemhez vagy szekcióhoz sem.

```
stickySectionHeadersEnabled={true}
```

```
<ListHeaderComponent={() => (  
  <Text style={{ fontSize: 24 }}>Bevásárlólista</Text>  
)}  
/>
```

```
<ListFooterComponent={() => (  
  <Text>Nincs több elem</Text>  
)}  
/>
```

VirtualizedList komponens

- A VirtualizedList egy lista komponens, amelyre a FlatList és a SectionList is épül.
- Teljes kontrollt ad az adatbetöltés és renderelés felett.

Adatkezelés:

- Nem közvetlenül tömböt vár, hanem függvényeken keresztül kéri le az adatokat:
 - getItem(data, index)
 - getItemCount(data)
- <https://reactnative.dev/docs/virtualizedlist>



VirtualizedList komponens

Megjelenítés módja:

```
const getItem = (_data, index) => ({
  id: Math.random().toString(12).substring(0),
  title: `Item ${index + 1}`,
});

const getItemCount = _data => 50;
```

```
<VirtualizedList
  initialNumToRender={4}
  renderItem={({item}) => <Item title={item.title} />}
  keyExtractor={item => item.id}
  getItemCount={getItemCount}
  getItem={getItem}
/>
```

Props neve	Szerepe
getItem	Egy elem lekérése
getItemCount	Elemszám meghatározása
renderItem	Elem megjelenítése
initialNumToRender	Megadja, hogy a lista kezdeti megjelenítéskor hány elemet rendereljen.
keyExtractor	Egyedi kulcs